

Project Documentation

AuditX is an enterprise **internal-audit platform** for a bank's Internal Audit function, deployed single-tenant inside the bank's network. It covers the full audit value chain — the audit universe and risk-based annual plan, checklist templates, audit engagements and fieldwork, findings and remediation, controls and compliance, sanctions, reporting, analytics, and audit-committee governance — on a hardened, on-premises stack.

Architecture · Stack · Data & security model · Local setup · Testing · Deployment · Conventions | For the module-by-module feature map, see the Module & Submodule Reference.

1 Technology stack

AREA	TECHNOLOGY
Backend	.NET 10 · ASP.NET Core 10 (controllers) · EF Core 10 (code-first, GUID keys) · SQL Server 2022
Caching / jobs	Redis 7 (session denylist, cache) · Hangfire (durable background jobs)
Messaging	RabbitMQ (integration events) · in-process domain-event dispatch
Auth	Active Directory (LDAPS + Kerberos/IWA), JWT session cookies · AD-over-REST gateway · Development provider locally
Frontend	Angular 21 (standalone components, signals) · Angular Material (skinned) · Tailwind v4 · SCSS · Lucide icons · PWA
Observability	Serilog structured logging · dependency health probes · append-only audit trail
Infra	Docker / Docker Compose · Nginx · GitHub Actions CI

2 Architecture

Clean Architecture with a strict dependency rule (outer depends on inner only):

AuditX.Domain	Entities, aggregates, value objects, domain events, invariants. No framework deps.
AuditX.Application	Use cases (CQRS commands/queries + handlers), DTOs, validation, port interfaces.
AuditX.Infrastructure	EF Core, repositories, identity providers, notification channels, jobs, external adapters.
AuditX.Api	ASP.NET Core controllers, auth filters, request contracts, composition root.

Key patterns and conventions:

- **CQRS** — a hand-rolled `IDispatcher` routes `ICommand` / `IQuery` to their handlers; `FluentValidation` runs in the pipeline. No MediatR.
- **DDD aggregates** — `AggregateRoot` / `Entity` bases, `Guard` / `DomainException` / `InvalidStateTransitionException` for invariants; child entities implement `IBelongsToAggregate` so their mutations promote the root's rowversion.
- **Optimistic concurrency** — every aggregate carries a `byte[]` rowversion; write handlers call `EnsureVersion( ... )` and return `409 Conflict` on a stale token (`RowVersionToken` base64 round-trips it).
- **Authorization** — permission-based (`[RequirePermission(key)]` + `IPermissionResolver`); resource-scoped checks are enforced inside handlers. Roles are AuditX-owned and independent of AD groups.
- **Audit trail** — an append-only activity log with before/after JSON snapshots, enforced at the database by a trigger; the `AuditingSaveChangesInterceptor` stamps created/updated audit fields.
- **Domain events** → **notifications** — post-commit events are serialised and handed to the M10 notification pipeline (rules → recipients → templated dispatch over Email/SMS/Teams) via a durable Hangfire job.
- **API envelope** — responses are wrapped { `data`, `...` }; enums serialise as `snake_case`; the SPA echoes the `version` token on every mutation for concurrency.

3 Repository layout

```
backend/
  AuditX.slnx
  src/
    AuditX.Domain/          domain model (Audits, Planning, Exceptions, Controls, Templates, ...)
    AuditX.Application/     CQRS handlers, DTOs, abstractions (ports)
    AuditX.Infrastructure/  EF Core persistence + migrations, identity, notifications, jobs
    AuditX.Api/             controllers, contracts, filters, Program.cs
  tests/
    AuditX.Domain.Tests/    aggregate/invariant unit tests
    AuditX.Application.Tests/ command/query handler + service unit tests
    AuditX.Infrastructure.Tests/ persistence/adaptor tests
    AuditX.Api.IntegrationTests/ end-to-end HTTP tests on real SQL Server (Testcontainers)
frontend/
  auditx-web/              Angular workspace (features/, core/, shared/)
  database/                SQL helper scripts (EF migrations live under Infrastructure)
  docker/                  Dockerfiles + nginx config
  docs/                    this documentation, ADRs, deployment runbooks
  .github/                  CI workflows
```

The Angular app mirrors the backend: `core/` (services, models, i18n, permissions, icons), `shared/` (layout, reusable components, directives), and one folder per **feature area** under `features/` (audits, planning, exceptions, controls, analytics, ac, reports, admin, ...).

4 Data & persistence

- **EF Core code-first**, GUID (v7) keys, snake_case tables/columns, enum-to-snake conversions.
- **61 migrations** applied automatically on API start when `Database:MigrateOnStartup` is enabled.
- **Seeding** — on startup the API seeds built-in roles, the permission catalogue, maker-checker gates, risk dimensions, rating scales, **response option sets**, reference data, and notification defaults (all idempotent). A richer demo dataset (populated audits, plan, universe, controls, findings) seeds when `Database:SeedDemoData=true`.

5 Security model

- No stored user passwords; authentication is delegated to Active Directory. Sessions are short-lived JWTs in HttpOnly cookies with a Redis denylist for revocation.
- Fine-grained **permissions** gate every endpoint; sensitive/resource-scoped operations re-check scope in the handler. The **Administrator** role holds the full catalogue.
- **Maker-checker** dual control gates configurable high-risk actions (e.g. template publish, MAP approval, sanctions-grid edit, config activation).
- **Integrity** — reports and AC packs are SHA-256 hash-verified; the audit trail is append-only and DB-enforced; evidence uploads are magic-byte + hash validated and re-verified on read.

6 Running locally

Option A — Docker Compose (everything)

```
docker compose up -d --build
```

The API auto-applies migrations and seeds data on startup. Ports come from the committed compose file (`8080` API / `4200` web); this machine's `docker-compose.override.yml` remaps them:

SERVICE	THIS MACHINE	DEFAULT
API	http://localhost:8085 (Swagger at <code>/swagger</code>)	8080
Web (SPA)	http://localhost:4288	4200
SQL Server	14330	1433
Redis	63790	6379
RabbitMQ	56720 / 15673 (mgmt)	5672 / 15672

After a code change, rebuild just the affected container, e.g. `docker compose up -d --build api` (auto-migrates) or `docker compose up -d --build web`.

Option B — Host

- 1. Dependencies: `docker compose up -d sqlserver redis rabbitmq`
- 2. Backend: `dotnet run --project backend/src/AuditX.Api`
- 3. Frontend: `cd frontend/auditx-web && npm install && npm start` (dev server on `:4222` proxying the API)

Demo users (Development identity provider)

Password `Passw0rd!` for all. `admin` (Administrator — all permissions), `manager` (Audit Manager), `auditor` (Auditor), `auditee` (Auditee — self-assessment, MAP submission).

7 Testing

SUITE	SCOPE
<code>AuditX.Domain.Tests</code>	aggregate invariants & state machines (unit)
<code>AuditX.Application.Tests</code>	command/query handlers & services (unit, NSubstitute)
<code>AuditX.Infrastructure.Tests</code>	persistence & adapters
<code>AuditX.Api.IntegrationTests</code>	~58 end-to-end HTTP flows against a real SQL Server via Testcontainers
<code>frontend/auditx-web</code>	509 Karma/Jasmine specs

```
# backend
cd backend && dotnet test AuditX.slnx
# frontend
cd frontend/auditx-web && npx ng test --watch=false --browsers=ChromeHeadless && npx ng build
```

Integration tests require Docker (Testcontainers spins up SQL Server + Redis). i18n has parallel `en.json` / `fr.json` dictionaries kept at strict key parity.

8 Deployment

Production is on-premises single-tenant. See the runbooks:

- **DEPLOYMENT_RUNBOOK.md** — on-prem deployment.
- **AZURE_DEPLOYMENT_RUNBOOK.md** — Azure Container Apps + IaC.
- **adr/** — architecture decision records.

Production must set the real identity provider (`Identity:Provider = ActiveDirectory` or `ActiveDirectoryRest`), the SMTP / SMS gateway / Teams webhook, and **disable** the Development provider and demo seeding.

9 Conventions

- **Commits** follow Conventional Commits (`feat(...)`, `fix(...)`, `test(...)`).
- **Frontend** — standalone components, signals, OnPush; Tailwind v4 + skinned Material; Lucide icons via `<app-icon name="...">`; every user-facing string localised (en/fr).
- **Backend** — one vertical slice per feature (domain → application → infrastructure → api → tests); new config is seeded idempotently; new domain events are added to the notification catalogue.